



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

PROJECT 2

SECP2753

SECTION 02

DATA MINING

LECTURER: DR. ROZILAWATI BINTI DOLLAH @ MD ZAIN

NO.	NAME	MATRIC NUMBER
1.	YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201
2.	ANIS SAFIYYA BINTI JANAI	A23CS0049
3.	SYARIFAH DANIA BINTI SYED ABU BAKAR	A23CS0183

1.0 Introduction

2.0 business and data understanding

2.1 problem formulation

2.2 dataset overview

3.0 overview and importance of preprocessing

3.1 data cleaning and preprocessing steps

3.2 results of data preparation

4.0 model development (unsupervised learning)

4.1 overview of unsupervised learning methods

4.2 applied algorithms and process workflow

4.3 model results

4.4 model performance and analysis

5.0 Conclusion

4.0 MODEL DEVELOPMENT (UNSUPERVISED LEARNING)

4.1 OVERVIEW OF UNSUPERVISED LEARNING METHODS

Unsupervised learning is a branch of machine learning that focuses on uncovering hidden patterns or grouping in data that does not have predefined labels. In this project, we utilize two prominent unsupervised learning algorithms, which is K-Means Clustering and DBSCAN (Density-Based Spatial Clustering of Applications with Noise) to analyze a dataset containing vehicle engine characteristics, fuel consumption, CO₂ emissions, and many more. Our main objective is to identify meaningful groupings within the data that can reveal valuable insights related to fuel efficiency and environmental impact. Then, the performance of the clustering was evaluated using the silhouette score which helped us to measure the quality of the CO₂ emissions clusters.

4.2 APPLIED ALGORITHMS AND PROCESS WORKFLOW

We created the unsupervised learning by using python. Below are the main steps in the process.

4.2.1 K-Means algorithm

Insight 1: Fuel efficiency segmentation

Data Preparation:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Step 1: Load Dataset
df = pd.read_excel(r"C:\Users\ybzah\Downloads\data mining project 2\dataset_DM3.xlsx")

# Step 2: Select Features
features = ['fuelconsumption_city', 'fuelconsumption_hwy',
            'fuelconsumption_comb', 'fuelconsumption_comb_mpg',
            'co2emissions']
X = df[features]

# Step 3: Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Model Creation:

```
# Step 4: Apply K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
df['Cluster'] = clusters

# Plot Fuel Efficiency Segmentation
fig, axes = plt.subplots(2, 2, figsize=(13, 10))
fig.suptitle("Fuel Efficiency Segmentation by Cluster", fontsize=16)

sns.scatterplot(data=df, x='fuelconsumption_city', y='co2emissions', hue='Cluster', palette='viridis', ax=axes[0, 0])
axes[0, 0].set_title('City Fuel Consumption vs CO2 Emissions')
axes[0, 0].set_xlabel('City Fuel Consumption (L/100km)')
axes[0, 0].set_ylabel('CO2 Emissions (g/km)')

sns.scatterplot(data=df, x='fuelconsumption_hwy', y='co2emissions', hue='Cluster', palette='viridis', ax=axes[0, 1])
axes[0, 1].set_title('Highway Fuel Consumption vs CO2 Emissions')
axes[0, 1].set_xlabel('Highway Fuel Consumption (L/100km)')
axes[0, 1].set_ylabel('CO2 Emissions (g/km)')

sns.scatterplot(data=df, x='fuelconsumption_comb', y='co2emissions', hue='Cluster', palette='viridis', ax=axes[1, 0])
axes[1, 0].set_title('Combined Fuel Consumption vs CO2 Emissions')
axes[1, 0].set_xlabel('Combined Fuel Consumption (L/100km)')
axes[1, 0].set_ylabel('CO2 Emissions (g/km)')

sns.scatterplot(data=df, x='fuelconsumption_comb_mpg', y='co2emissions', hue='Cluster', palette='viridis', ax=axes[1, 1])
axes[1, 1].set_title('Fuel Efficiency (MPG) vs CO2 Emissions')
axes[1, 1].set_xlabel('Combined Fuel Efficiency (MPG)')
axes[1, 1].set_ylabel('CO2 Emissions (g/km)')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

Model Evaluation:

```
# Show Cluster Centroids
centroids_original = scaler.inverse_transform(kmeans.cluster_centers_)
centroids_df = pd.DataFrame(centroids_original, columns=features)
centroids_df.index = [f'Cluster {i}' for i in centroids_df.index]
print("\n=== Cluster Centroid Summary (Original Units) ===")
print(centroids_df.round(2))

# Evaluate Clustering Performance
sil_score = silhouette_score(X_scaled, clusters)
db_score = davies_bouldin_score(X_scaled, clusters)

print(f"Silhouette Score: {sil_score:.4f}")
print(f"Davies-Bouldin Index: {db_score:.4f}")
```

Step 1: Data Preparation

- We imported the dataset and select 5 relevant features for clustering: fuelconsumption_city, fuelconsumption_hwy, fuelconsumption_comb, fuelconsumption_comb_mpg, and co2emissions
- The selected features were standardized using StandardScaler.

Step 2: Visualization

- The K-Means algorithm was applied to group the data into three clusters based on similarity in fuel consumption and CO₂ emission features.
- It initializes the K-Means clustering model with `n_clusters=3` and `random_state=42`
- Four scatter plots were generated to show the relationship between various fuel consumption metrics and CO₂ emissions.

Step 3: Evaluation metrics

- All data was selected to computer the Silhouette Score and Davies-Bouldin index:
Silhouette Score: 0.64
Davies-Bouldin Score: 0.49

Insight 2: Engine characteristics vs emissions

Data Preparation:

```
# Step 1: Data Preparation
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score
import matplotlib.pyplot as plt
import seaborn as sns

# Load your dataset
df = pd.read_excel(r"C:\Users\ybzah\Downloads\data mining project 2\dataset_DM3.xlsx")

# Select relevant features
features = df[['engine_size', 'cylinders', 'co2emissions']]

# Scale the data
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)
```

Model creation:

```
# Step 2: K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(scaled_features)
df['Cluster'] = clusters
```

```

# Step 3: Centroids in Original Units
centroids_scaled = kmeans.cluster_centers_
centroids_original = scaler.inverse_transform(centroids_scaled)
centroids_df = pd.DataFrame(centroids_original, columns=['ENGINE_SIZE', 'CYLINDERS', 'CO2EMISSIONS'])
print("Centroids in original units:\n", centroids_df)

# Step 4: Scatter Plot - Engine Size vs CO2 Emissions
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='engine_size', y='co2emissions', hue='Cluster', palette='Set2', s=60)
plt.title('Engine Size vs CO2 Emissions')
plt.xlabel('Engine Size')
plt.ylabel('CO2 Emissions')
plt.grid(True)
plt.legend(title='Cluster')
plt.show()

# Step 5: Scatter Plot - Cylinders vs CO2 Emissions
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='cylinders', y='co2emissions', hue='Cluster', palette='Set2', s=60)
plt.title('Cylinders vs CO2 Emissions')
plt.xlabel('Cylinders')
plt.ylabel('CO2 Emissions')
plt.grid(True)
plt.legend(title='Cluster')
plt.show()

```

Model Evaluation:

```

# Step 6: Evaluation Metrics
sil_score = silhouette_score(scaled_features, df['Cluster'])
db_score = davies_bouldin_score(scaled_features, df['Cluster'])

print(f"\nSilhouette Score: {sil_score:.3f}")
print(f"Davies-Bouldin Index: {db_score:.3f}")

```

Step 1: Data Preparation

- We imported the dataset and selected 3 relevant features for clustering: engine_size, cylinder, co2emissions. The selected features were standardized using StandardScaler.

Step 2: Visualization

- The K-Means algorithm was applied to group the data into three clusters based on similarity in engine size, cylinder and CO₂ emission features.
- It initializes the K-Means clustering model with n_clusters=3 and random_state=42
- Two scatter plots were generated to show the relationship between various car characteristics and CO₂ emissions.

Step 3: Evaluation metrics

- All data was selected to compute the Silhouette Score and Davies-Bouldin index:
Silhouette Score: 0.67
Davies-Bouldin Score: 0.52

4.2.2 DBSCAN algorithm

Insight 1: Fuel efficiency segmentation

Data Preparation:

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Define separate analyses
fuel_features = {
    "City": "fuelconsumption_city",
    "Highway": "fuelconsumption_hwy",
    "Combined": "fuelconsumption_comb",
    "Combined MPG": "fuelconsumption_comb_mpg"
```

Model Creation:

```
for label, fuel_col in fuel_features.items():
    print(f"\n🔍 Clustering on {label} Fuel Consumption")

    # Step 1: Select features
    features = ['engine_size', fuel_col, 'co2emissions']
    clustering_df = merged_df[features].dropna().copy()

    # Step 2: Normalize the data
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(clustering_df)

    # Step 3: Apply DBSCAN
    dbscan = DBSCAN(eps=0.8, min_samples=10)
    clustering_df['Cluster'] = dbscan.fit_predict(X_scaled)

    # Step 4: Visualize
    plt.figure(figsize=(8, 6))
    sns.scatterplot(
        data=clustering_df,
        x=fuel_col,
        y='co2emissions',
        hue='Cluster',
        palette='tab10'
    )
    plt.title(f"DBSCAN: {label} Fuel Consumption vs CO2 Emissions")
    plt.xlabel(f"{label} Fuel Consumption")
    plt.ylabel("CO2 Emissions (g/km)")
    plt.legend(title='Cluster')
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

```
# Step 5: Print cluster distribution
print("Cluster label counts:")
print(clustering_df['Cluster'].value_counts().sort_index())
```

Model Evaluation:

```
# Step 6: Evaluate clustering quality (only if clusters exist)
if clustering_df['Cluster'].nunique() > 1 and -1 in clustering_df['Cluster'].values:
    mask = clustering_df['Cluster'] != -1
    silhouette = silhouette_score(X_scaled[mask], clustering_df['Cluster'][mask])
    db_index = davies_bouldin_score(X_scaled[mask], clustering_df['Cluster'][mask])
    print(f"→ Silhouette Score: {silhouette:.3f}")
    print(f"→ Davies-Bouldin Index: {db_index:.3f}")
else:
    print("→ Not enough valid clusters to compute performance scores.")
```

Step 1: Data Preparation

- We selected 5 relevant features for clustering: fuelconsumption_city, fuelconsumption_hwy, fuelconsumption_comb, fuelconsumption_comb_mpg and co2emissions.
- Each fuel-related feature was used separately along with engine size and emissions to create multiple clustering scenarios.
- The data was then normalized using StandardScaler to ensure consistent scale across all features.

Step 2: Visualization

- The DBSCAN algorithm was applied to group vehicles based on their fuel consumption patterns and emission output.
- This density-based method does not require defining the number of clusters and can detect outliers.

- For each fuel type (city, highway, combined, and combined MPG), a scatter plot was generated to visualize the clusters using that feature vs CO₂ emissions.
- Each plot revealed consistent structure: one large cluster, one medium group, and a few small clusters — with outliers (label -1) showing extreme cases.

Step 3: Evaluation metrics

Insight 2: Engine characteristics vs emissions

Data Preparation:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN

# Step 1: Select relevant features
features = ['engine_size', 'cylinders', 'co2emissions']
clustering_df = merged_df[features].dropna().copy()

# Step 2: Normalize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(clustering_df)
```

Model creation:

```

# Step 3: Apply DBSCAN
dbscan = DBSCAN(eps=0.8, min_samples=10)
clustering_df['Cluster'] = dbscan.fit_predict(X_scaled)

# Step 4a: Visualize Engine Size vs CO2 Emissions
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=clustering_df,
    x='enginesize',
    y='co2emissions',
    hue='Cluster',
    palette='Set2'
)
plt.title("DBSCAN: Engine Size vs CO2 Emissions")
plt.xlabel("Engine Size (L)")
plt.ylabel("CO2 Emissions (g/km)")
plt.legend(title='Cluster')
plt.grid(True)
plt.tight_layout()
plt.show()

# Step 4b: Visualize Cylinders vs CO2 Emissions
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=clustering_df,
    x='cylinders',
    y='co2emissions',
    hue='Cluster',
    palette='Set1'
)

```

```
plt.title("DBSCAN: Cylinders vs CO2 Emissions")
plt.xlabel("Cylinders")
plt.ylabel("CO2 Emissions (g/km)")
plt.legend(title='Cluster')
plt.grid(True)
plt.tight_layout()
plt.show()

# Step 5: Print cluster distribution
print("Cluster label counts:")
print(clustering_df['Cluster'].value_counts().sort_index())
```

Model Evaluation:

```
mask2 = clustering_df['Cluster'] != -1
silhouette2 = silhouette_score(X_scaled[mask2], clustering_df['Cluster'][mask2])
db_index2 = davies_bouldin_score(X_scaled[mask2], clustering_df['Cluster'][mask2])
print(f"Insight 2 - Silhouette Score: {silhouette2:.3f}")
print(f"Insight 2 - Davies-Bouldin Index: {db_index2:.3f}")
```

Step 1: Data Preparation

- We selected 3 relevant features for clustering: enginesize, cylinders, and co2emissions .
- These features were chosen because they directly reflect a vehicle's engine capacity and are expected to influence CO₂ emissions.
- The selected features were then standardized using StandardScaler to ensure equal contribution to the distance calculation in DBSCAN.

Step 2: Visualization

The DBSCAN algorithm was applied to group vehicles based on similarities in engine characteristics and emission levels.

Unlike K-Means, DBSCAN does not require specifying the number of clusters in advance and is capable of identifying noise or outlier points.

Two scatter plots were generated:

- Engine Size vs CO₂ Emissions
- Cylinders vs CO₂ Emissions

These visualizations helped highlight the clusters formed and revealed vehicles with extreme values as outliers (label -1).

Step 3: Evaluation metrics

-

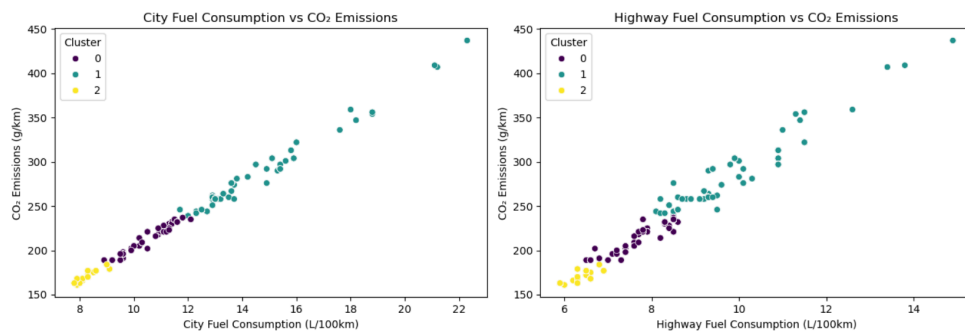
4.3 MODEL RESULTS

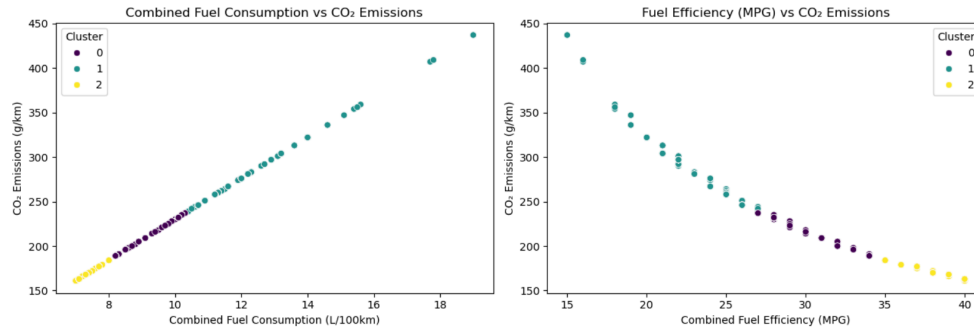
Results for both algorithm

4.3.1 Result for K-Means Algorithm

Insight 1:

Fuel Efficiency Segmentation by Cluster





```

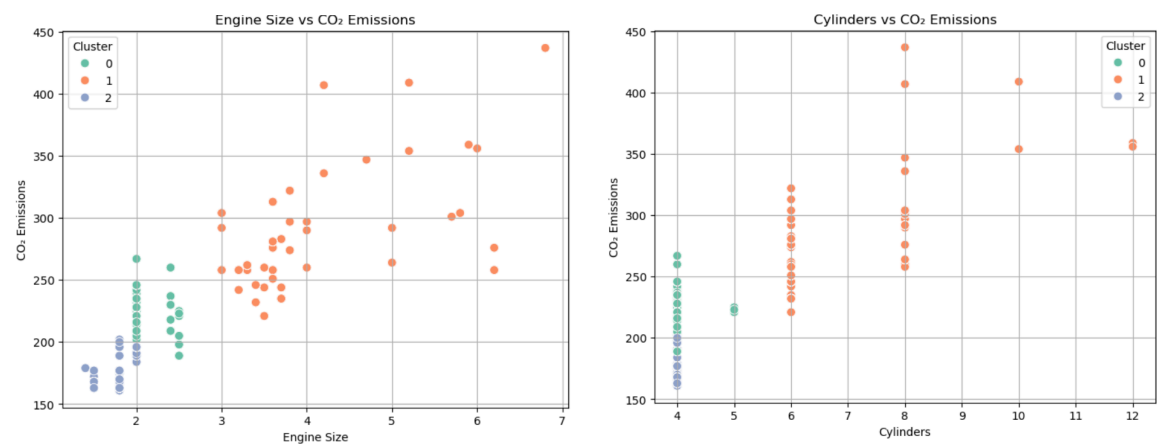
=== Cluster Centroid Summary (Original Units) ===
      fuelconsumption_city fuelconsumption_hwy fuelconsumption_comb \
Cluster 0                10.09                7.60                8.98
Cluster 1                13.56                9.66                11.80
Cluster 2                 8.19                 6.36                 7.33

      fuelconsumption_comb_mpg co2emissions
Cluster 0                   31.48         206.64
Cluster 1                   24.12         271.49
Cluster 2                   38.45         168.76
Silhouette Score: 0.6393
Davies-Bouldin Index: 0.4923

```

Based on the results, in most of the graph, cluster 1(Teal) is spread out toward the upper of the graph. This shows that these cars are least fuel efficient as it produces more CO₂ emissions. Meanwhile the cluster 2(Yellow), which is spread out mostly at the bottom shows that it is the most fuel efficient, as these cars have the lowest fuel consumption. The cluster 0 (Purple) sits in between, as it shows that these cars are of moderate fuel efficiency.

Insight 2:



Centroids in original units:

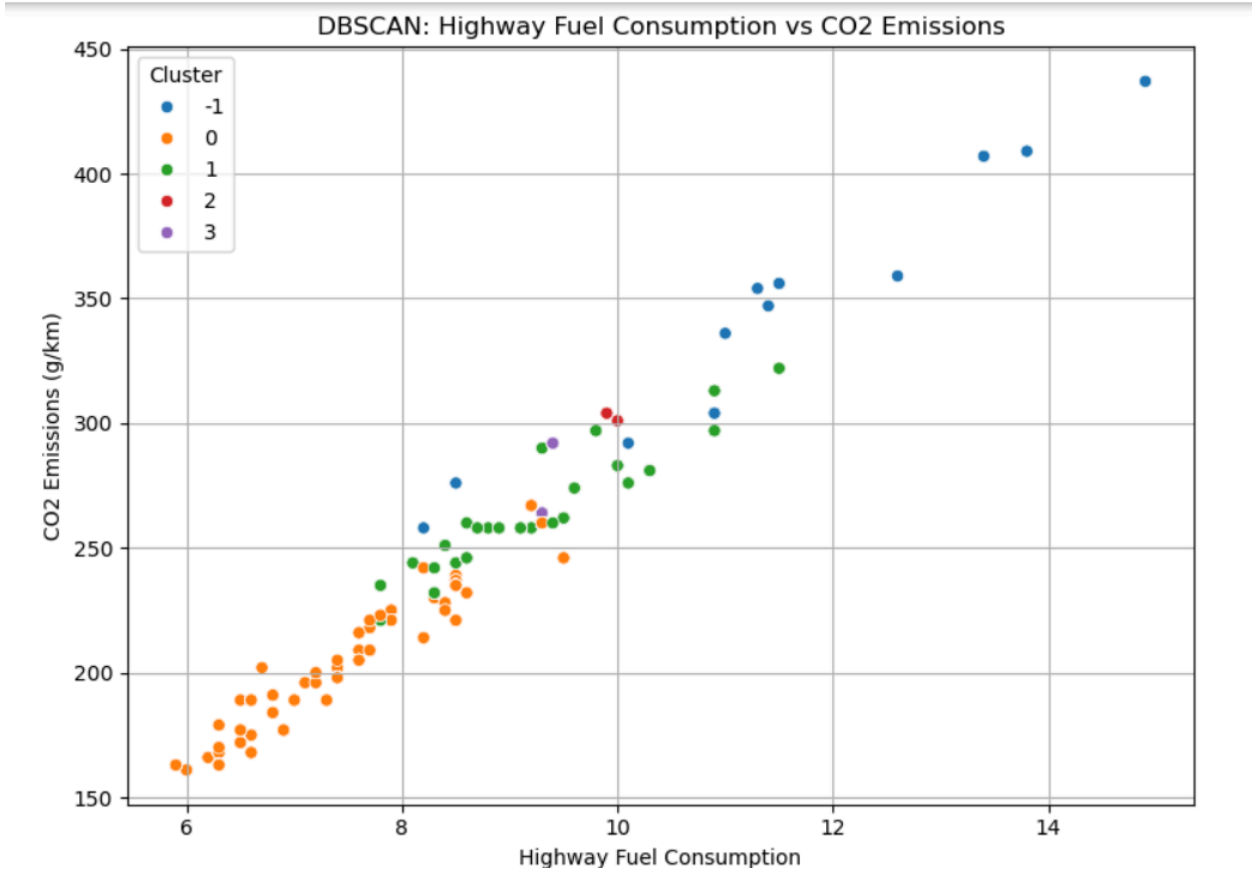
	ENGINE SIZE	CYLINDERS	CO2EMISSIONS
0	2.307121	4.002297	218.310490
1	3.575977	6.092834	268.864007
2	1.782651	4.000000	170.868578

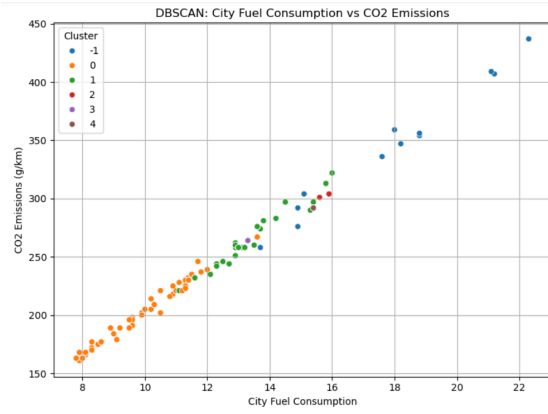
Silhouette Score: 0.673

Davies-Bouldin Index: 0.522

Based on the graph of Engine Size vs CO₂ it shows that small engines have low emissions that are indicated by blue dots, then mid size engines are equal to moderate emissions which are indicated by green, and large engines produced high emissions are indicated by orange colour. The graph Cylinders vs CO₂ shows that 4-Cylinder Cars (likely lower emissions) are grouped as eco-friendly or smaller cars. 6 or 8-Cylinder Cars (with higher emissions) is Grouped as high-performance or utility vehicles.

4.3.2 Result for DBSCAN Algorithm





4.4 MODEL PERFORMANCE AND ANALYSIS

To evaluate the quality of the clustering algorithm, two important internal validation metrics are used which is Silhouette Score and Davies-Bouldin Index.

Silhouette Score: measures how similar each data point is to its own cluster compared to other clusters. A higher silhouette value indicates that the data points are well matched to their own group and clearly separated from others

Davies-Bouldin Index: measures the average similarity between each cluster and the one most similar to it. A lower Davies-Bouldin value suggests better clustering with more compact and well-separated groups.

4.4.1 Performance and Analysis of Insight 1

The model successfully groups the cars into 3 distinct clusters based on their Fuel Consumption and CO2 emissions. The clustering result then was being evaluated by using two internal validation metrics. Below is the result:

	Silhouette Score	Davies-Bouldin Index
K-Means	0.64	0.49
DBSCAN		

For K-Means, the silhouette score is near to 1.0 which indicates that the clustering in the data is good. In addition, the Davies-Bouldin Index value is 0.49, which indicates that Points in the same cluster are close to each other and different clusters are far apart from one another.

4.4.1 Performance and Analysis of Insight 2

The model successfully groups the cars into 3 distinct clusters based on their cars characteristics and CO2 emissions. The clustering result then was being evaluated by using two internal validation metrics. Below is the result:

	Silhouette Score	Davies-Bouldin Index
K-Means	0.67	0.52
DBSCAN		

For K-Means, the silhouette score is near to 1.0 which indicates that the clustering in the data is good. In addition, the Davies-Bouldin Index value is 0.52, which indicates that Points in the same cluster are close to each other and different clusters are far apart from one another.